

AID: A Framework for AI Infrastructure Dynamics

Abi Aryan

Joule (joule.lat)

Correspondence: res@abiaryan.com

Abstract

A useful representation of AI inference infrastructure must preserve the state variables and interactions that determine future service under changing operating conditions. We introduce AID (AI Infrastructure Dynamics), a framework for representing AI infrastructure as a coupled system spanning physical, computational, networking, and serving processes. The formulation represents system state, observations, actions, disturbances, persistent configuration, and service outcomes, while allowing structured and variable-size state, asynchronous observations, multiple physical timescales, and demand that responds to service. We distinguish representations that support forecasting under an existing policy from those that preserve service-relevant behavior under changed actions, and separate both from identification of intervention responses. Two analytical results characterize a prediction-error lower bound under observational ambiguity and a sufficient condition for exact controlled state reduction. These results apply established information and state-abstraction principles to the AID formulation. We then define validation criteria for evaluating whether candidate representations preserve information relevant to cache structure, workload history, measurement availability, and imposed actions. This paper presents the AID formulation and validation criteria; it does not report empirical facility-level validation.

1 Introduction

A request’s latency depends on both external demand and the infrastructure’s current ability to serve it. That ability may depend on cache structure, request progress, network contention, resource allocation, and physical conditions that aggregate telemetry does not fully describe. Dynamic KV-memory management, iteration-level scheduling, and thermal- and power-aware serving illustrate how these layers interact [8, 20, 17].

We introduce AID (AI Infrastructure Dynamics), a framework for representing AI infrastructure as a coupled system. AID formalizes the plant through a state spanning physical, computational, networking, and serving processes, together with the observations available to an observer, the actions applied to the system, external disturbances, persistent configuration and operating constraints, and service outcomes. The state may contain structured and variable-size objects, and its evolution may occur across multiple physical timescales and operating regimes.

This representation provides a basis for asking what information a learned state must preserve. A representation may predict future behavior accurately under the policy that generated its training data while omitting information needed when routing, batching, resource allocation, power control, or workload changes. Prediction under one policy therefore does not by itself establish that the representation preserves the dynamics relevant to changed actions.

Figure 1 summarizes the system boundary and its cross-layer interactions.

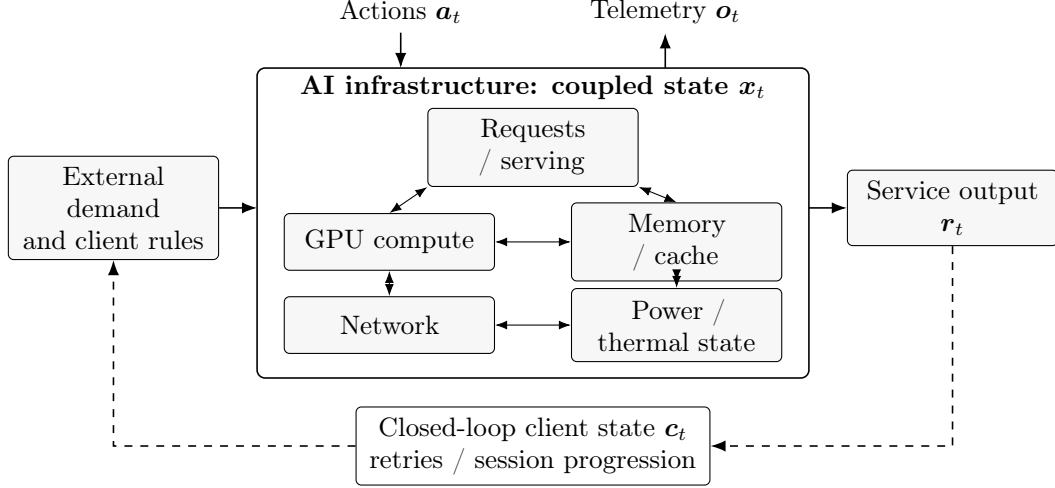


Figure 1: The AID system boundary. Solid arrows show demand, actions, observations, service, and schematic cross-layer coupling, not an exhaustive dependency graph or a serial execution pipeline. The dashed path represents service-dependent arrivals through client state; it is absent in open-loop replay. External intent and client rules are distinguished from realized demand.

The central learning question is:

Which history-based representations preserve the distributions relevant to future service under admissible interventions?

This question connects the AID representation to partially observed sequential decision-making [6], state estimation [16], controlled state abstraction [2, 10], and causal identification [13]. We do not prescribe a particular predictive architecture. Instead, we specify the system representation, information boundary, policy class, and service criteria under which a candidate representation can be evaluated.

We develop the formulation in four parts:

1. **A coupled state-space formulation** for structured state, physical time, information availability, and demand that responds to service.
2. **A sufficiency framework** distinguishing forecasting, controlled-state reduction, and intervention identification for a specified policy class and available information.
3. **Two analytical diagnostics** characterizing observational ambiguity and exact controlled reduction, including a simple cache example.
4. **A validation protocol** connecting the formulation to controlled comparisons of state representations, observations, workloads, and imposed actions.

Our contribution is to formulate an explicit dynamical representation for AI infrastructure and to specify criteria for evaluating whether a learned representation preserves the information relevant to prediction and changed actions. The framework does not introduce a new estimator, representation-learning algorithm, or empirically validated facility model; instead, it provides the system representation and analytical criteria needed to evaluate such models.

2 Related Work

2.1 Partial Observation and State Estimation

Belief states in partially observable Markov decision processes summarize uncertainty from action and observation histories under a specified model [6]. Bayesian filtering and smoothing provide methods for estimating latent state under different assumptions, including linear-Gaussian models, nonlinear approximations, and particle-based methods [16].

We use this distinction between telemetry and latent state while also asking whether the chosen system boundary, available observations, and assumed dynamics are adequate. Computing a state estimate does not by itself show that the state can be inferred uniquely or that the dynamics are identifiable. A representation may support prediction under one policy while omitting information needed after an intervention.

2.2 Controlled State Abstraction

Givan et al. develop equivalence notions and model minimization for Markov decision processes using stochastic bisimulation [2]. Li et al. distinguish classes of state abstraction according to the properties they preserve [10]. These results motivate evaluating a compressed state by the decisions it is intended to support, rather than only by reconstruction accuracy.

In Section 5, we apply this reduction principle to the joint distribution of the next state and service output. Preserving this joint distribution lets us compare future service under a common policy. The proof uses a standard induction argument, rather than introducing a new bisimulation theorem.

2.3 Inference-Serving Mechanisms

Inference infrastructure is not limited to autoregressive language-model serving. Clockwork builds a distributed DNN-serving system around predictable execution and request-level latency objectives [3]. It provides an example of how execution models, resource management, and scheduling interact to produce service outcomes. Its evaluated setting differs from autoregressive LLM serving, where evolving request progress and cache state introduce additional representation questions.

Orca’s iteration-level scheduling motivates request progress and evolving batch membership as serving-state variables [20]. vLLM implements PagedAttention to manage KV memory in blocks, connecting serving decisions to allocation and sharing state [8]. SGLang’s RadixAttention organizes cached prefixes for reuse, making prefix identity and structure relevant beyond total occupancy [21]. Sarathi-Serve uses chunked prefill to coordinate prompt processing and token generation, illustrating why execution phase and scheduling history matter to service [1].

2.4 Disaggregation, Placement, and Execution Resources

DistServe separates prefill and decoding across GPUs and chooses phase-specific resource allocation, parallelism, and placement with communication bandwidth in view [22]. Splitwise likewise separates the phases across machines, using their different compute, memory, and power characteristics to design homogeneous and heterogeneous clusters [12]. These mechanisms motivate a state that records which phase owns the work, where that work executes, and how state transfer interacts

with available resources. Phase-dependent demand is not by itself evidence of a latent regime with distinct transition dynamics.

Mooncake treats KV data as a distributed resource, using CPU, DRAM, SSD, and network resources for a disaggregated cache alongside separate prefill and decoding clusters [14]. Together, SGLang, Mooncake, and DistServe motivate three related state questions: what is cached, where it resides, and which execution phase and resource currently own the work [21, 14, 22]. A representation intended to support changed routing or placement must be tested against these distinctions, rather than assumed sufficient because it predicts aggregate occupancy.

TensorRT-LLM provides a production-oriented implementation reference for optimized inference, including in-flight batching, paged attention, and multi-GPU or multi-node execution [11]. We use this documentation to establish the implementation context, not as evidence for a mathematical guarantee or a comparative performance claim.

A recent survey by Jiang et al. organizes system-aware KV-cache optimization into temporal execution and scheduling, spatial placement and migration, and structural representation and retention [5]. This taxonomy situates the mechanisms above without replacing their primary references. AID asks a different question: which information must a representation retain to preserve future service under specified actions and observations? These systems expose complementary parts of that state/action/dynamics problem; they are not presented as incorrect abstractions or as validation of a single universal state representation.

2.5 Physical Coupling and Feedback

TAPAS studies thermal- and power-aware scheduling for LLM inference, while DynamoLLM considers dynamic inference-cluster configuration under performance and energy objectives [17, 18]. These systems demonstrate that computational decisions can interact with physical infrastructure variables.

Work on metastable distributed-system failures identifies amplification mechanisms and persistent low-goodput overload [4]. We therefore include client retries and service feedback when they affect the workload within the chosen system boundary. This connection motivates a test for persistent overload; it does not show that a particular inference workload is metastable.

2.6 Temporal Structure, Interventions, and Constraints

Queueing analysis and empirical network-traffic studies provide antecedents for modelling demand through variability and temporal dependence rather than a stationary average alone [7, 9]. Causal inference distinguishes observational conditioning from imposed interventions [13]. Model predictive control provides one possible decision layer, but feasibility and stability claims require controller-specific assumptions [15].

We bring these distinctions together by specifying the infrastructure state, available information, disturbance process, and service criterion. Results from queueing, causal inference, or control apply only when their assumptions also hold for the infrastructure model.

3 The AID Formulation

AID represents AI infrastructure as a dynamical system with state, actions, external disturbances, persistent configuration, observations, and service outputs:

$$\mathcal{S} = (\mathcal{X}, \mathcal{A}, \mathcal{D}, \Theta, \mathcal{O}, \mathcal{R}, f, h, g). \quad (1)$$

Here, $\mathbf{x}_t \in \mathcal{X}$ denotes the system state at time t , $\mathbf{a}_t \in \mathcal{A}$ an action applied to the system, $\mathbf{d}_t \in \mathcal{D}$ an external disturbance, $\mathbf{o}_t \in \mathcal{O}$ the telemetry available to an observer, and $\mathbf{r}_t \in \mathcal{R}$ the resulting service output. The parameter $\boldsymbol{\theta} \in \Theta$ denotes persistent configuration, such as hardware, software, or deployment choices that remain fixed over the period being modeled.

For systems that admit a numerical state representation, the formulation can be written in the familiar state-space form

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{a}_t, \mathbf{d}_t; \boldsymbol{\theta}) + \boldsymbol{\epsilon}_t, \quad (2)$$

$$\mathbf{o}_t = h(\mathbf{x}_t; \boldsymbol{\theta}) + \boldsymbol{\nu}_t, \quad (3)$$

$$\mathbf{r}_t = g(\mathbf{x}_t, \mathbf{a}_t, \mathbf{d}_t; \boldsymbol{\theta}) + \boldsymbol{\xi}_t. \quad (4)$$

The terms $\boldsymbol{\epsilon}_t$, $\boldsymbol{\nu}_t$, and $\boldsymbol{\xi}_t$ represent process, observation, and service-output uncertainty, respectively. A discrete operating regime z_t can also be included in the state, allowing the transition law to depend on the current regime, for example through f_{z_t} .

This numerical form is a shorthand rather than a restriction on the state representation. In particular, AID allows $\mathbf{x}_t$ to contain structured or variable-size objects such as request sets, queues, cache structures, placement relations, and other state that cannot naturally be represented by a fixed-dimensional vector. External client inputs are treated separately because, depending on the system boundary, they may be exogenous disturbances or may themselves depend on the service produced by the infrastructure.

3.1 Structured States and Transition Laws

The state space need not be $\mathbb{R}^n$ or a smooth manifold. It can contain numerical physical variables together with variable-length request records, cache trees, block-placement relations, queue structures, and other combinatorial state. Cache allocation, prefix reuse, and distributed KV placement provide concrete motivation for retaining structure beyond numerical occupancy [8, 21, 14]. Disaggregated execution also motivates recording phase ownership and the resources involved in transferring request state [22, 12].

For general measurable state spaces, we describe evolution with a transition law over valid states:

$$\mathbf{x}_{t+1} \sim K_{\Delta_t}(\cdot \mid \mathbf{x}_t, \mathbf{a}_t, \mathbf{d}_t; \boldsymbol{\theta}), \quad \Delta_t = \tau_{t+1} - \tau_t, \quad (5)$$

where τ_t denotes physical time.

We treat a vector embedding as a proposed reduction of the state, whose adequacy must be tested. Operations such as insertion, eviction, and request progression change structured objects and need not have the form of additive Gaussian perturbations. Physical transients likewise require a noise model appropriate to the process being described.

We assume throughout that state, action, history, and output spaces are standard Borel spaces, and that the mappings and kernels are measurable. These assumptions allow us to define regular

conditional distributions. We interpret conditional laws almost everywhere with respect to the corresponding history distribution. We use additive-noise notation only when vector addition is well defined and the resulting state remains admissible.

3.2 Clock and Information Semantics

A multi-rate model updates components at declared physical intervals or events. Slow states can evolve between measurements; holding their estimates constant does not make the underlying physical states static. This distinction is important when fast processes influence slower variables. For example, transient computational activity may alter power and heat generation before a lower-rate thermal measurement records the resulting change. Timescale separation therefore does not by itself justify eliminating fast transients. Multi-rate numerical integration is an established computational approach [19], but selecting an integration scheme does not establish that a coarse representation preserves service-level quantities such as latency tails.

We denote the information available at physical time τ_t by the random history record $\mathcal{I}_t$. This record contains previous actions and sensor records received by that time, together with measurement times, validity information, and aggregation windows. Here $\mathcal{I}_t$ is a random record, not a sigma-algebra.

We obtain a candidate learned state from this record through a mapping ϕ :

$$\mathbf{s}_t = \phi(\mathcal{I}_t; \boldsymbol{\theta}). \quad (6)$$

This definition limits the representation to information available when the decision is made. A trace aligned after an experiment may include measurements that the online controller had not yet received. We therefore distinguish retrospective analysis from online state estimation. When uncertainty about latent state is decision-relevant, the learned representation may instead be a belief distribution. A point estimate need not preserve the uncertainty required for future decision-making [6].

Figure 2 separates the latent state, available history, and candidate representation used for prediction and decision-making.

3.3 Demand Boundary

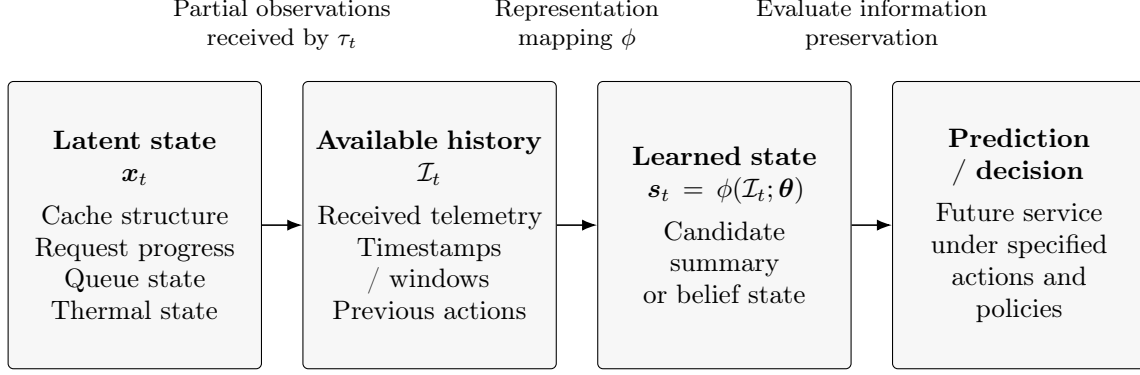
In open-loop replay, arrivals are supplied without response to service. In closed-loop operation, client state and declared external inputs may instead determine realized demand.

Let $\mathbf{c}_t$ denote client-side predictive state and $\mathbf{e}_t$ declared external inputs:

$$\mathbf{d}_t^{\text{work}} \sim \rho(\cdot \mid \mathbf{c}_t, \mathbf{e}_t), \quad (7)$$

$$\mathbf{c}_{t+1} \sim K_c(\cdot \mid \mathbf{c}_t, \mathbf{r}_t, \mathbf{e}_t). \quad (8)$$

Retries and response-dependent session progression motivate this feedback extension. Related amplification mechanisms have been studied in distributed systems [4]. To obtain a Markov representation, we include client memory and any other disturbance history needed to determine the next-state distribution. Whether $\mathbf{e}_t$ is exogenous depends on the chosen system boundary. When comparing policies, we match external intent and client rules, rather than requiring identical realized arrivals. The arrivals may differ because each policy produces a different service response, which in turn changes subsequent client behavior.



Current state is not directly available; history contains only records received by τ_t .

Figure 2: From latent infrastructure state to a candidate learned representation. The observation arrow summarizes a history of partial measurements, not a direct observation of the current state. The mapping ϕ uses only available information. Prediction under changed actions requires testing what the representation preserves; the diagram does not establish sufficiency or identify intervention responses from logs.

4 Representation Sufficiency and Learning Tasks

Given an AID representation, we define the learning task for a fixed configuration, a finite horizon H , an available information record, a class of admissible policies Π_{adm} , and a disturbance law. Policies are non-anticipating: an action can depend on information already available, but not on future observations. For a controlled reduction, we compare policies that use only the retained representation and output history, or prescribe the same action sequence. If a policy can use information omitted from the representation, it may choose different actions. That would change the comparison as well as the future service distribution.

Let $\mathbf{Y}_H$ collect future observations and declared service quantities. For a policy π , let $\mathcal{L}^\pi(\mathbf{Y}_H | \mathcal{I}_t)$ denote the corresponding conditional law under the specified model.

An intervention-relevant representation seeks

$$\mathcal{L}^\pi(\mathbf{Y}_H | \mathcal{I}_t) \approx \mathcal{L}^\pi(\mathbf{Y}_H | \mathbf{s}_t) \quad \text{for every declared } \pi \in \Pi_{\text{adm}}. \quad (9)$$

To make this approximation testable, we specify how the two distributions are compared, the permitted error, the prediction horizon, and the workloads and operating conditions over which the comparison applies. This requirement is stronger than prediction under a single logging policy. A representation may accurately forecast future behavior under the policy used to collect its training data while discarding information needed when routing, batching, resource allocation, or power control changes.

We distinguish three objectives according to what information the representation must preserve and what can be inferred from the data:

Forecast sufficiency: preserving future quantities under a specified observational or logging policy.

Controlled-state sufficiency: preserving the future service-relevant distribution under a declared class of admissible actions and policies.

Intervention identification: establishing the response to an intervention from available information under assumptions sufficient to justify that inference.

These objectives are related but not interchangeable. A representation can be forecast-sufficient without being controlled-state sufficient, and a predictive model can exist without the corresponding intervention response being identifiable from observational data [13]. In particular, $\mathcal{L}^\pi$ denotes a law under the specified model, not automatically a causal law identified from logs.

Candidate learning tasks include history-to-belief estimation [16], structured-state compression evaluated against controlled abstraction criteria [2, 10], conditional trajectory modelling, and calibrated service-risk prediction.

For models intended to predict interventions, we evaluate both workloads and actions not used in training. Random splits of a trace collected under one policy do not directly test whether the model predicts the response to a different action. We also specify the service quantities used for training and evaluation. Mean latency, tail events, rejections, failures, energy consumption, and thermal excursions describe different aspects of service. An accurate prediction of one quantity does not by itself establish accuracy for the others.

5 Elementary Analytical Diagnostics

We use two analytical results to characterize what information a representation can and cannot preserve. The first gives a prediction-error lower bound when the available history cannot distinguish models with different intervention responses. It follows from the total-variation triangle inequality. The second gives a sufficient condition for reducing a Markov state while preserving future service under a common policy, following established state-abstraction principles [2, 10]. These results provide tests of the assumptions behind an infrastructure representation. We apply established principles to the AID formulation rather than claim new general theorems in control or state abstraction.

5.1 Ambiguity Limits Common Predictions

For probability measures P, Q , define

$$d_{\text{TV}}(P, Q) = \sup_B |P(B) - Q(B)|, \quad (10)$$

where the supremum is over measurable sets.

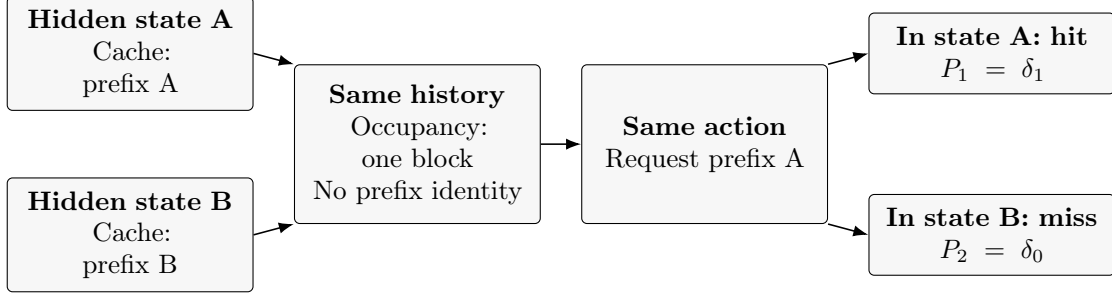
Proposition 1 (Prediction error under observational ambiguity). *Suppose two admissible models induce the same history law μ for $\mathcal{I}_t$. Under the same declared future intervention, let $P_1(\cdot | i)$ and $P_2(\cdot | i)$ be regular conditional laws for a common future-output variable, and let $Q(\cdot | i)$ be a common prediction kernel. Define the measurable functions*

$$\gamma(i) = d_{\text{TV}}(P_1(\cdot | i), P_2(\cdot | i)), \quad (11)$$

$$\eta_j(i) = d_{\text{TV}}(P_j(\cdot | i), Q(\cdot | i)), \quad j \in \{1, 2\}. \quad (12)$$

Then, for μ -almost every i ,

$$\max\{\eta_1(i), \eta_2(i)\} \geq \frac{\gamma(i)}{2}. \quad (13)$$



For the binary hit indicator: $d_{TV}(P_1, P_2) = 1$, so any common predictor has worst-case error at least $1/2$.

Figure 3: Observational ambiguity in a deterministic cache example. The two admissible hidden states have the same occupancy-only history but different responses to the same imposed action. Here δ_1 and δ_0 are point masses for a binary cache-hit indicator. The action is prescribed in both states; equal telemetry does not imply equal response. This is a mathematical construction, not an empirical cache result.

Moreover,

$$\max_{j \in \{1,2\}} \int \eta_j(i) \mu(di) \geq \frac{1}{2} \int \gamma(i) \mu(di). \quad (14)$$

Proof. For the chosen versions of the conditional laws, the triangle inequality gives

$$\gamma(i) \leq \eta_1(i) + \eta_2(i) \quad (15)$$

almost everywhere. At least one term is therefore at least $\gamma(i)/2$. Integrating the same inequality gives

$$\int \gamma d\mu \leq \int \eta_1 d\mu + \int \eta_2 d\mu, \quad (16)$$

so at least one expected error is at least half the integrated separation. $\square$

The interpretation is an information limit. If the available information cannot distinguish two admissible models that respond differently to the same intervention, then a predictor that assigns the same conditional distribution to that information must incur nonzero error for at least one model whenever the integrated separation is positive.

This does not imply that similar telemetry snapshots establish observational equivalence. Additional sensors, longer histories, or sufficiently informative interventions may distinguish the models. The result instead identifies a condition under which the available information is insufficient.

As a simple illustration, suppose telemetry records only cache occupancy. Two latent cache states can have identical occupancy and observed history while containing different prefixes. Under a common action requesting one of those prefixes, a binary cache-hit output can have different conditional laws in the two states. In the deterministic construction, one state produces a hit and the other a miss, giving $\gamma = 1$ and a worst-case total-variation prediction error of at least $1/2$. The example is purely mathematical: it illustrates observational ambiguity rather than making an empirical claim about a particular cache implementation.

Figure 3 illustrates this construction and the bound in Proposition 1.

5.2 Exact Controlled Reduction

Let X_t be a Markov state of the augmented facility–client system at a fixed configuration, including clocks, predictive memory, and latent regimes as necessary. Here X_t , A_t , and R_t denote the random state, action, and service output represented elsewhere by $\mathbf{x}_t$, $\mathbf{a}_t$, and $\mathbf{r}_t$. Let ψ be a measurable reduction to a standard Borel space.

We describe the next reduced state and current service output through their joint one-step distribution under an imposed action. This structural kernel incorporates the specified external-input law and is defined for every admissible state–action pair, including pairs absent from the logging data. The conditional notation in the proposition refers to this structural law, rather than a response inferred automatically from logs.

Proposition 2 (Exact controlled reduction). *Suppose there exists a kernel $\bar{K}$ such that, for every full state x , admissible action a , and measurable sets B, C ,*

$$\Pr\{\psi(X_{t+1}) \in B, R_t \in C \mid X_t = x, A_t = a\} = \bar{K}(B \times C \mid \psi(x), a). \quad (17)$$

Then, under any common non-anticipating policy using only reduced-state and output history, started with the same policy memory, two initial states with the same reduction generate the same joint reduced-state and service-output trajectory law at every finite horizon.

Proof. The initial reduced states and policy memories agree. Given the same reduced history, the common policy selects the same action distribution. The assumed kernel then gives the same conditional distribution for the next reduced state and service output. Integrating over the common history establishes equality of the one-step extension; induction gives equality of the joint trajectory law at every finite horizon. $\square$

The condition requires the joint law of the next reduced state and service output to depend on the current state only through $\psi(x)$ and the imposed action. It is closely related to established state-equivalence and abstraction principles [2, 10]; the induction proof applies these principles rather than introducing a new bisimulation theorem.

The condition is stronger than preserving next-state means or aggregate occupancy. It requires the relevant joint transition and service-output distribution to agree for every admissible state–action pair, including pairs not encountered under the logging policy. It is therefore a sufficient condition for controlled reduction, not a claim that such a reduction can be identified from observational data alone. The comparison uses the specified disturbance law and common policy memory; policies that use hidden full-state information fall outside its scope.

6 Validation Protocol

The formulation leads to a set of controlled tests of whether candidate representations preserve the information required for prediction and decision-making under declared interventions. The protocol is intended to test the assumptions and distinctions introduced by AID; no empirical results are reported here.

Candidate comparisons include:

- explicit cache structures versus occupancy-only and enriched summaries;

- instantaneous arrival descriptors versus temporal histories;
- timestamp-aware online estimates versus retrospectively aligned features;
- point state estimates versus uncertainty-aware representations;
- open-loop workload replay versus response-dependent clients;
- observational forecasts versus forecasts under imposed actions.

Experiments should preserve request and infrastructure versions, reset conditions, measurement availability, and offered-request accounting. Forecast tests should be separated from imposed-action tests. A model that predicts future behavior under the logging policy has not thereby demonstrated that it predicts the response to a changed action.

Calibration should be evaluated using both coverage and sharpness across the declared horizon and operating domain. Negative results should identify which intervention, workload, or operating regime breaks a proposed reduction rather than being absorbed into a single aggregate score.

A reference environment can make the learning task reproducible without establishing universal adequacy. Evaluation across other configurations is required before claiming transfer. Regime switching and retry persistence remain hypotheses until supported by controlled evidence.

7 Scope and Limitations

This paper presents a formulation and validation methodology rather than a learned representation, estimator, or empirically validated facility model. The relationship between a candidate representation and the state of a real infrastructure system must be established through measurement and controlled evaluation. The exact reduction condition in Section 5 is a sufficient condition, not a practical claim that real infrastructure admits an exact low-dimensional representation. Approximate reductions require an explicit discrepancy measure, prediction horizon, and error tolerance, together with analysis of how error changes over the horizon.

The ambiguity result depends on the model class and information available to the observer. Similar telemetry does not by itself establish observational equivalence, and additional measurements or interventions may resolve an apparent ambiguity. Intervention identification requires assumptions or experimental evidence sufficient to support the corresponding causal claim [13]. The framework does not by itself guarantee identification from observational logs.

Likewise, AID does not establish service-level objective (SLO) guarantees, energy optimality, or safe control. Such claims require explicit service populations, disturbance laws, constraints, and controller-specific analysis [15]. Finally, the adequacy of any representation may depend on hardware, workloads, client behavior, and operating regime. A representation validated under one configuration should not be assumed to transfer to another without evidence.

8 Conclusion

AID formulates the problem of learning representations for AI infrastructure as a question of information preservation under future service and admissible actions. The framework distinguishes

telemetry from latent state, forecasting from imposed-action prediction, supplied arrivals from response-dependent demand, and numerical embeddings from structured state. The central question is not whether a representation reconstructs the measurements that are easiest to log. It is whether the representation preserves the information required to predict future service under the decisions the system may actually take.

The resulting formulation provides explicit objects, assumptions, analytical diagnostics, and validation criteria for investigating that question. It does not assume that commonly exposed infrastructure metrics constitute a sufficient state, nor that predictive accuracy under an existing policy establishes intervention-relevant adequacy. AID therefore provides a basis for making representation claims in AI infrastructure explicit, testable, and falsifiable.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*, OSDI 24, pages 117–134. USENIX Association, 2024. <https://www.usenix.org/conference/osdi24/presentation/agrawal>.
- [2] Robert Givan, Thomas Dean, and Matthew Greig. Equivalence notions and model minimization in Markov decision processes. *Artificial Intelligence*, 147(1–2):163–223, 2003. [https://doi.org/10.1016/S0004-3702\(02\)00376-4](https://doi.org/10.1016/S0004-3702(02)00376-4).
- [3] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like Clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation*, OSDI ’20, pages 443–462. USENIX Association, 2020. <https://www.usenix.org/conference/osdi20/presentation/gujarati>.
- [4] Lexiang Huang, Matthew Magnusson, Abishek Bangalore Muralikrishna, Salman Estyak, Rebecca Isaacs, Abutalib Aghayev, Timothy Zhu, and Aleksey Charapko. Metastable failures in the wild. In *16th USENIX Symposium on Operating Systems Design and Implementation*, OSDI ’22, pages 73–90. USENIX Association, 2022. <https://www.usenix.org/conference/osdi22/presentation/huang-lexiang>.
- [5] Jiantong Jiang, Peiyu Yang, Rui Zhang, and Feng Liu. Towards efficient large language model serving: A survey on system-aware KV cache optimization. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 38450–38476. Association for Computational Linguistics, 2026. <https://doi.org/10.18653/v1/2026.findings-acl.1916>.
- [6] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. [https://doi.org/10.1016/S0004-3702\(98\)00023-X](https://doi.org/10.1016/S0004-3702(98)00023-X).
- [7] J. F. C. Kingman. On queues in heavy traffic. *Journal of the Royal Statistical Society: Series B (Methodological)*, 24(2):383–392, 1962. <https://doi.org/10.1111/j.2517-6161.1962.tb00465.x>.

- [8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, pages 611–626. Association for Computing Machinery, 2023. <https://doi.org/10.1145/3600006.3613165>.
- [9] Will E. Leland, Murad S. Taqqu, Walter Willinger, and Daniel V. Wilson. On the self-similar nature of Ethernet traffic (extended version). *IEEE/ACM Transactions on Networking*, 2(1):1–15, 1994. <https://doi.org/10.1109/90.282603>.
- [10] Lihong Li, Thomas J. Walsh, and Michael L. Littman. Towards a unified theory of state abstraction for MDPs. In *Proceedings of the Ninth International Symposium on Artificial Intelligence and Mathematics*, 2006. Author-hosted paper: Full text.
- [11] NVIDIA. TensorRT-LLM: Overview. Official project documentation, 2026. Accessed October 3, 2026. Official documentation.
- [12] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture*, ISCA, 2024. Author-hosted paper: Full text.
- [13] Judea Pearl. Causal inference in statistics: An overview. *Statistics Surveys*, 3:96–146, 2009. <https://doi.org/10.1214/09-SS057>.
- [14] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies*, FAST '25, pages 155–170. USENIX Association, 2025. <https://www.usenix.org/conference/fast25/presentation/qin>.
- [15] James B. Rawlings, David Q. Mayne, and Moritz M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 edition, 2022. Fourth electronic printing, April 2022. Author-hosted text.
- [16] Simo Särkkä and Lennart Svensson. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2 edition, 2023. <https://doi.org/10.1017/9781108917407>.
- [17] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Esha Choukse, Haoran Qiu, Rodrigo Fonseca, Josep Torrellas, and Ricardo Bianchini. TAPAS: Thermal- and power-aware scheduling for LLM inference in cloud platforms. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, volume 2 of *ASPLOS '25*, pages 1266–1281. Association for Computing Machinery, 2025. <https://doi.org/10.1145/3676641.3716025>.
- [18] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture*, HPCA, pages 1348–1362. IEEE, 2025. <https://doi.org/10.1109/HPCA61900.2025.00102>.
- [19] SUNDIALS Project. ARKODE: Introduction. Official SUNDIALS documentation, version 7.9.0. Accessed October 3, 2026. Official documentation.

- [20] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, OSDI '22, pages 521–538. USENIX Association, 2022. <https://www.usenix.org/conference/osdi22/presentation/yu>.
- [21] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, 2024. <https://doi.org/10.52202/079017-2000>.
- [22] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, OSDI '24, pages 193–210. USENIX Association, 2024. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.